

Universidad Autónoma de Chiriquí

Facultad de Economía

Escuela De Ciencias de Computación



Lic. en Gestión de Tecnología de la Información

¨Programación V PROG 309¨

Integrante:

Daniela Pittí

Jairo Chavez

Débora Cisneros

Prof. Alexis Flores

Trabajo Escrito

Laboratorio 3(Uber)

Fecha de Entrega

09/09/2026

II Semestre

1. INTRODUCCIÓN AL CASO

Uber es una de las plataformas de movilidad más grandes del mundo, operando en más de 70 países y procesando millones de viajes diariamente. Detrás de su sencilla interfaz móvil se esconde una de las arquitecturas de software más complejas y sofisticadas de la industria tecnológica.

Para poder manejar esta escala sin precedentes, Uber no podía depender de una arquitectura monolítica tradicional ni de sistemas de comunicación simples. La solución que adoptaron fue una **arquitectura basada en eventos (EDA)** utilizando **Apache Kafka** como su bus de eventos central.

Este bus de eventos permite que los más de 100 microservicios de Uber se comuniquen de manera **asíncrona y desacoplada**, lo que significa que ningún servicio depende directamente de otro para funcionar. Esto le da a Uber la capacidad de escalar, actualizar y mantener su sistema sin interrumpir el servicio global.

2. ¿CÓMO FUNCIONA EL BUS DE EVENTOS EN UBER?

2.1. El Problema que Resuelve

Imagina que estás pidiendo un Uber. En segundos, tu solicitud debe:

1. Llegar a los conductores cercanos
2. Asignar el mejor conductor
3. Calcular la ruta y el precio
4. Procesar tu método de pago
5. Enviarte notificaciones en tiempo real
6. Actualizar el mapa con la ubicación del conductor
7. Guardar datos para análisis posteriores

Todo esto debe ocurrir en milisegundos y para millones de usuarios simultáneamente. Si cada uno de estos pasos dependiera del anterior, el sistema sería increíblemente lento y frágil.

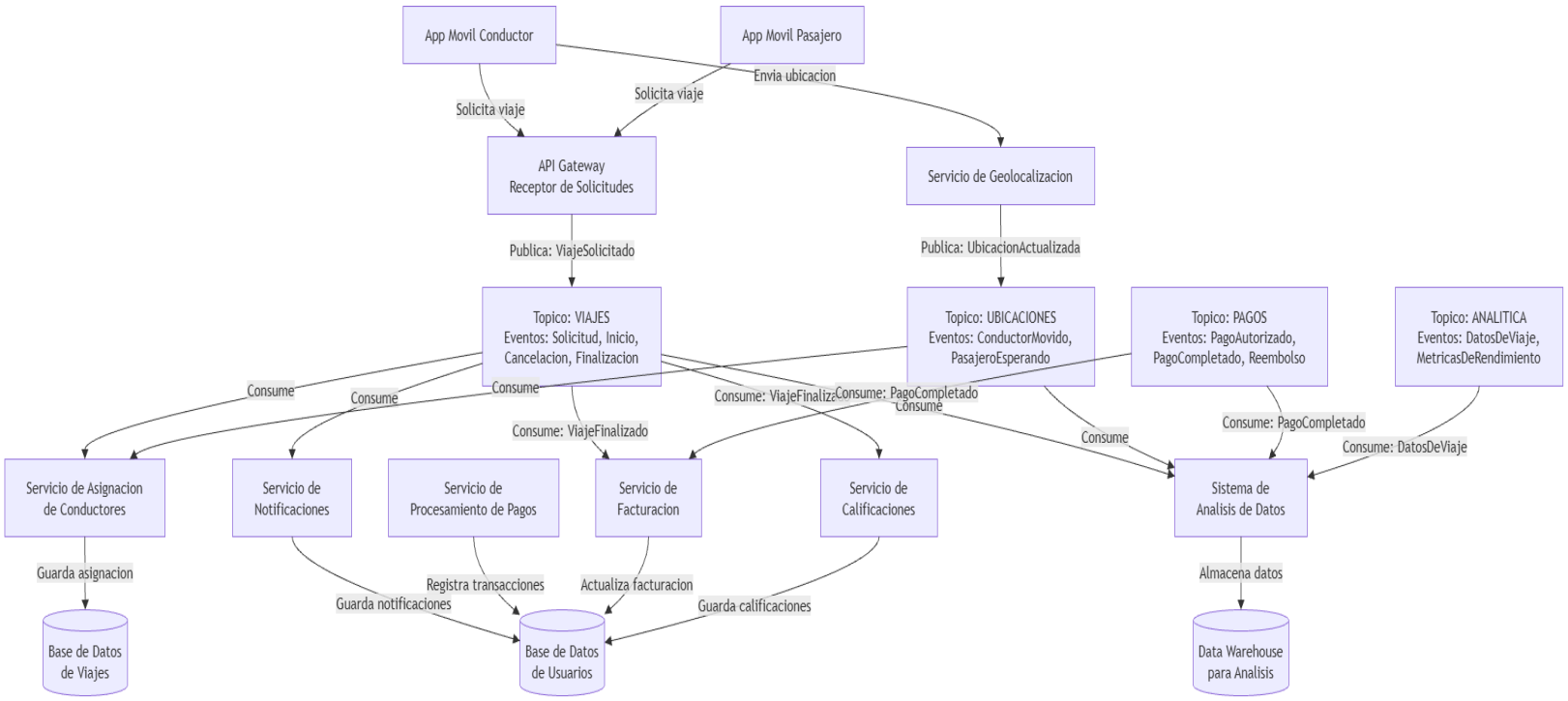
2.2. La Solución: Eventos y Tópicos

Uber resuelve esto mediante un sistema de **eventos**. Un evento es simplemente un mensaje que dice "algo pasó". Por ejemplo:

- "ViajeSolicitado" (cuando un usuario pide un auto)
- "UbicaciónActualizada" (cuando el conductor se mueve)
- "PagoCompletado" (cuando se procesa el pago)

Estos eventos se publican en **tópicos** (como canales de televisión) dentro de Kafka. Los servicios que necesitan esa información se **suscriben** a esos tópicos y reciben los eventos automáticamente.

3. DIAGRAMA DE COMPONENTES



4. EXPLICACIÓN DEL DIAGRAMA

4.1. Componentes y sus Funciones

Componente	*Rol en el sistema*	*Tipo*
App Móvil (Conductor/Pasajero)	Interfaz de usuario. Genera solicitudes de viaje y envía ubicaciones.	Productor de eventos
API Gateway	Punto de entrada único. Recibe solicitudes HTTP y las convierte en eventos.	Productor de eventos
Servicio de Geolocalización	Procesa datos de GPS de conductores y pasajeros en tiempo real.	Productor de eventos

Tópico: VIAJES	Canal que contiene todos los eventos relacionados con el ciclo de vida del viaje.	Bus de eventos
Tópico: UBICACIONES	Canal con actualizaciones constantes de posicionamiento.	Bus de eventos
Tópico: PAGOS	Canal que maneja transacciones financieras y autorizaciones.	Bus de eventos
Tópico: ANALÍTICA	Canal para datos de rendimiento y métricas del sistema.	Bus de eventos
Servicio de Asignación	Escucha solicitudes de viaje y ubicaciones para emparejar conductores.	Consumidor de eventos
Servicio de Notificaciones	Envía alerts, actualizaciones y recordatorios a usuarios.	Consumidor de eventos
Servicio de Procesamiento de Pagos	Autoriza y procesa transacciones con tarjetas o billeteras digitales.	Consumidor de eventos
Sistema de Análisis de Datos	Recoge eventos para generar reportes, mejorar algoritmos y predecir demanda.	Consumidor de eventos
Servicio de Facturación	Genera facturas y recibos después de cada viaje.	Consumidor de eventos
Servicio de Calificaciones	Procesa ratings de conductores y pasajeros.	Consumidor de eventos

4.2. Flujo de Comunicación

1. **Solicitud de viaje**: El pasajero abre la app y pide un auto. La API Gateway recibe esto y publica un evento **"ViajeSolicitado"** en el tópico **VIAJES**.
2. **Asignación de conductor**: El Servicio de Asignación está escuchando el tópico VIAJES. Cuando recibe el evento de solicitud, consulta el tópico UBICACIONES para ver qué conductores están cerca. Selecciona al mejor y publica un evento **"ConductorAsignado"** de vuelta al tópico VIAJES.
3. **Notificación al usuario**: El Servicio de Notificaciones también está escuchando VIAJES. Cuando ve el evento **"ConductorAsignado"**, envía un push notification al pasajero diciéndole "Tu conductor está en camino".
4. **Procesamiento de pago**: Cuando el viaje termina, el Servicio de Asignación publica un evento **"ViajeFinalizado"**. Esto hace que el Servicio de Procesamiento de Pagos se active, autorice el cargo y publique un evento **"PagoCompletado"** en el tópico PAGOS.
5. **Análisis de datos**: Todo el tiempo, el Sistema de Análisis de Datos está consumiendo eventos de todos los tópicos para alimentar dashboards, mejorar predicciones de demanda y optimizar rutas.

5. VENTAJAS CONCRETAS DEL BUS DE EVENTOS EN UBER

5.1. Desacoplamiento Total

Cada servicio en Uber es independiente. El Servicio de Asignación no necesita saber cómo funciona el de Pagos, ni viceversa. Solo necesitan entender el formato de los eventos que publican o consumen. Esto permite que equipos diferentes trabajen en servicios distintos sin pisarse.

5.2. Tolerancia a Fallos (Resiliencia)

Si el Servicio de Notificaciones falla durante 5 minutos, los eventos de solicitudes de viaje se acumulan en Kafka. Cuando el servicio vuelve, retoma el procesamiento desde donde lo dejó y envía todas las notificaciones pendientes. El viaje en sí nunca se interrumpe.

5.3. Escalabilidad Horizontal

Si en hora punta el Servicio de Asignación se satura, Uber puede agregar más instancias de ese servicio. Todas ellas consumirán del mismo tópico VIAJES, repartiéndose la carga automáticamente.

5.4. Flexibilidad para Nuevas Funcionalidades

Cuando Uber quiso lanzar Uber Eats, simplemente crearon nuevos servicios que se suscribieran a los tópicos existentes (como ubicaciones y pagos) y publicaran eventos nuevos (como "PedidoEntregado"). Ningún servicio antiguo tuvo que ser modificado.

5.5. Manejo de Picos de Demanda

Durante eventos como Año Nuevo o conciertos masivos, el número de solicitudes de viaje puede multiplicarse por 10. Kafka actúa como un buffer que absorbe estos picos. Los servicios consumidores procesan los eventos a su propio ritmo sin que el sistema colapse.

6. RESPUESTAS A LAS PREGUNTAS DE ANÁLISIS

Pregunta 1: ¿Por qué el patrón elegido es más adecuado que los otros dos?

El bus de eventos es el rey para sistemas como Uber porque **ningún servicio necesita esperar a otro**. En el patrón P2P, todos los nodos son iguales, pero Uber tiene servicios muy especializados: unos hacen pagos, otros asignan conductores, otros mandan notificaciones. No tendría sentido que todos sean iguales. El broker es más simple, pero es como un cartero que solo entrega cartas; necesita saber a quién entregarle cada mensaje. En cambio, el bus de eventos es como un sistema de megafonía: cualquiera puede hablar y cualquiera puede escuchar, sin que el que habla sepa quién está escuchando.

La verdadera magia del bus de eventos en Uber es que permite que cada servicio vaya a su propio ritmo. El de pagos puede ir más lento si hay mucho volumen, pero el de asignación de conductores no se ve afectado. Eso es simplemente imposible con los otros patrones.

Pregunta 2: ¿Qué pasaría si el bus de eventos falla? ¿Hay respaldo?

¡Uy, esto es crítico! Si Kafka se muere, Uber se muere. No hay viajes, no hay pagos, no hay nada. Literalmente la app se queda tonta porque ningún servicio sabe lo que está pasando.

Pero Uber no es tonta. Han blindado el sistema con capas y capas de protección:

- **Replicación**: Cada mensaje se copia en al menos 3 servidores diferentes. Si uno explota, los otros tienen la información exacta.
- **Clústeres regionales**: No ponen todos los huevos en la misma canasta. Tienen centros de datos en América, Europa y Asia. Si un terremoto tumba un centro, los otros siguen funcionando.
- **Monitoreo 24/7**: Tienen un sistema llamado *Chaperone* que es como un guardia de seguridad que revisa que ningún mensaje se pierda. Si algo huele mal, las alarmas suenan y los ingenieros saltan en segundos.
- **Failover automático**: Si un servidor de Kafka falla, otro toma su lugar automáticamente. Los servicios ni siquiera se enteran del cambio.

Entonces, aunque el bus de eventos es el cerebro de Uber, es un cerebro con muchos respaldos para que si una neurona falla, las otras tomen el control.

Pregunta 3: Si usara P2P, ¿cómo se garantizaría la seguridad?

Aunque Uber no usa P2P, podemos imaginar cómo garantizarían la seguridad en un sistema descentralizado. Sería todo un reto porque no hay una autoridad central que valide las transacciones.

Para que funcione, tendrían que implementar algo similar a lo que hace BitTorrent o las blockchains:

- **Firmas digitales**: Cada mensaje llevaría la firma del nodo que lo envía. Si alguien intenta falsificar un mensaje, la firma no coincide y todos lo detectan. Es como una firma autógrafa que no se puede falsificar.
- **Consenso distribuido**: Para validar un viaje, varios nodos tendrían que ponerse de acuerdo. Algo como: "Para confirmar que el pasajero subió al auto, al menos 5 nodos cercanos deben testificar que vieron subir al pasajero". Es como tener varios testigos.
- **Sistema de reputación**: Los nodos que mienten o hacen trampa serían marcados y eventualmente aislados. Es como en un juego online donde los que hacen trampa son baneados.
- **Redundancia**: En lugar de confiar en un solo nodo, pedirías la misma información a varios. Si 5 de 6 nodos te dicen lo mismo, es probable que sea cierto.

Pero siendo sinceros, para la escala de Uber, todo esto sería demasiado lento y complicado. Por eso prefieren el bus de eventos, que es más simple y controlable.

Pregunta 4: Ventajas concretas frente a comunicación directa

Ventaja 1 - Resiliencia: Con comunicación directa, si el Servicio de Notificaciones se cae, el de Asignación se queda esperando la confirmación de que se envió la notificación y todo se traba. Con el bus de eventos, el de Asignación publica "ViajeSolicitado" y sigue su camino. El de Notificaciones consume ese evento cuando puede y envía la notificación después. Nadie espera a nadie.

Ventaja 2 - Escalabilidad: Si el de Asignación está saturado, simplemente creas 3 copias más y las pones a consumir del mismo tópico. Con comunicación directa, tendrías que modificar cada servicio que se comunica con él para que sepa que ahora hay 4 instancias. Es un dolor de cabeza.

Ventaja 3 - Nuevas funcionalidades: Cuando Uber lanzó Uber Eats, los nuevos servicios solo tuvieron que suscribirse a los tópicos de ubicaciones y pagos. No tocaron ni una línea del código existente. Con comunicación directa, tendrían que haber modificado decenas de servicios para que se comuniquen con los nuevos.

Ventaja 4 - Picos de demanda: En hora punta, todos los servicios reciben millones de eventos. Kafka actúa como una esponja que absorbe el exceso. Los servicios procesan a su velocidad sin preocuparse por el volumen. Con comunicación directa, el más lento se convierte en el cuello de botella y todo el sistema se ralentiza.

Ventaja 5 - Mantenimiento: Puedes actualizar o reiniciar un servicio sin afectar a los demás. Mientras el servicio esté caído, los eventos se acumulan en Kafka. Al volver, los procesa y se pone al día. Con comunicación directa, un servicio caído puede tumbar a varios que dependen de él.

6. Conclusiones

Investigar el caso de Uber permitió comprender que el patrón de bus de eventos resulta especialmente útil cuando muchos componentes de un sistema necesitan reaccionar a la misma información sin depender directamente unos de otros. A diferencia del broker y del P2P, este patrón centraliza la distribución de eventos en un componente que puede replicarse para evitar puntos únicos de falla, lo que explica por qué plataformas que operan a gran escala, como Uber, lo adoptan para coordinar sus microservicios. Esta investigación conecta directamente con lo visto en la clase teórica de la Semana 3 sobre cómo los patrones arquitecturales resuelven el acoplamiento entre componentes de formas distintas según las necesidades del sistema.

7. Referencias

Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium.

Blancarte, O. (2020). Introducción a la arquitectura de software.

Uber Engineering. Kafka on Kubernetes and uReplicator. eng.uber.com

Apache Kafka Documentation. kafka.apache.org